

Multilingual Voice Assistant

Interim Presentation

Ebin John Joseph · ASI24MCA-2025

Guide: Sukrith Lal · March 2026

~15-18 min · 19 slides

01 Title Slide

30s

Good morning. I'm **Ebin John Joseph**, register number ASI24MCA-2025, presenting my main project interim report – a **Hyper-Localized Multilingual Voice Assistant**. This project is guided by **Sukrith Lal sir**.

02 Introduction

1 min

India has **22 official languages** and over 19,500 dialects. Yet today's voice assistants primarily serve English and Hindi speakers. That leaves **780 million Indians** unable to use voice tech in their own language.

Three core problems:

1. **Digital divide** – 90% of India's internet users prefer local languages but services are English-first.
2. **Privacy** – every voice query to Alexa or Google is sent to a remote server.
3. **Connectivity** – 65% of rural India has unreliable internet.

Our project addresses all three – a **fully offline, privacy-preserving voice assistant** supporting **Malayalam** and 10 other Indian languages.

Main objective: Design and develop a hyper-localized multilingual voice assistant with edge-computing capabilities and agentic AI workflows.

Five specific objectives:

1. **Multilingual ASR** – 85%+ accuracy for Malayalam. We achieved **97%** with Pingala V1.
2. **Bidirectional translation** – ML↔EN with 90%+ accuracy via IndicTrans2.
3. **Agentic AI** – LangGraph routes queries to specialized agents.
4. **Edge readiness** – under 3 seconds end-to-end latency.
5. **Privacy-preserving** – all processing local, no data leaves the device.

We studied six existing systems:

Google Assistant & Alexa – cloud-only, Hindi only, no Malayalam, privacy concerns.

Rasa – local NLU but no ASR, TTS, or Indian language models.

IndicTrans2 – great translation (22 languages) but no voice pipeline or agents.

Whisper – 99 languages but lower accuracy for low-resource languages.

Bhashini – government cloud APIs, no offline/edge support.

Research gap: No system combines multilingual Indic ASR + local LLM + agentic workflows + edge deployment. That's what we built.

05 Proposed System

1 min

An end-to-end voice AI pipeline running **entirely on-device**.

Key features: fully offline, **11 Indian languages**, LangGraph agents, real-time dashboard.

Advantages: **zero cloud dependency**, no API fees, local inference (<3s latency), extensible architecture.

Core components: **Pingala V1** (2.94% WER), **IndicTrans2-Dist** (211M params), **Qwen3** via llama.cpp, **MMS-TTS**, **LangGraph** with 4 agents.

06 System Architecture

1 min

Three-layer architecture:

Input layer – voice audio in any supported language.

Processing layer – ASR → Translation (ML→EN) → Intent Detection → LangGraph Agent → LLM → Translation (EN→ML).

Output layer – MMS-TTS converts text to speech audio.

Agent system: LangGraph orchestrator routes to **Info Agent**, **Task Agent**, **Chat Agent**, and **Smart Home Agent**.

Agile Scrum – 6 sprints, 12 weeks, **114 story points**, 8 epics.

- Sprint 1: Research, ASR & TTS PoCs – **21 pts**
- Sprint 2: Translation, LLM, memory – **21 pts**
- Sprint 3: Pipeline, API, Pingala, WebSocket – **21 pts**
- Sprint 4: LangGraph agents, dashboard, DB – **18 pts**
- Sprint 5 (current): Edge optimization, Docker – **18 pts**
- Sprint 6 (planned): Testing & docs – **15 pts**

Currently at **87% completion** – 99/114 points.

Use case diagram – User interacts through voice queries, language selection, task management. System handles ASR, translation, intent, agent routing, TTS.

Activity diagram – audio captured → ASR → translation → intent → agent routing → LLM → translation back → TTS → audio response.

Level 1 DFD – voice flows through 5 stages: ASR, Translation, NLU/Intent, Agent/LLM, TTS. Data store holds conversations, metrics, cached responses.

ER Diagram – 3 entities:

- **Conversation** – session ID, language, timestamps, message count
- **Message** – role, content, intent, agent, timing. FK to Conversation (1:N)
- **SystemMetric** – component latency, memory, success rates (independent)

Pingala V1 for ASR – 2.94% WER, 50% better than Whisper for Indian languages.

IndicTrans2-Dist – 211M params, 5x smaller than full model, same accuracy.

Qwen3 + llama.cpp – GGUF quantized, efficient CPU inference, zero API cost.

LangGraph over vanilla chains – stateful graphs, conditional routing, memory.

FastAPI + WebSocket, **React** + **TypeScript** + **Tailwind** dashboard.

11 Sprint Progress

1 min

Sprints 1–4: **100% complete** – 81/81 story points.

Sprint 5 (active, 18 pts): Originally Raspberry Pi deployment. Pivoted to **edge-ready optimization** due to budget – response caching, model optimization, Docker with resource limits, benchmarking suite.

Sprint 6 (weeks 11–12): Unit tests, integration tests, documentation, final demo.

12 Implementation

1.5 min

Pipeline output: User speaks Malayalam asking about weather → ASR **480ms** → Translation **120ms** → Intent detection → InfoAgent + Qwen3 **340ms** → Translation back **130ms** → TTS **380ms**. **Total: 1.45 seconds.**

Dashboard – real-time monitoring: queries, latency breakdown, component health.

API – REST endpoints for transcribe, process, TTS, text-only + WebSocket for streaming.

13 Current Status

1 min

87% complete – 99/114 story points done, 15 remaining.

All core modules **100%**: ASR, translation, LLM, TTS, pipeline, agents, dashboard.

Edge optimization: **70%** – caching & Docker done, benchmarking in progress.

Testing & docs: **30%** – planned for Sprint 6.

14 Challenges

1 min

Memory constraints – 10–16 GB needed for all models simultaneously. Solved with **lazy loading** + distilled models + GGUF quantization.

Async integration – Pipecat async vs ML sync. Solved with **ThreadPoolExecutor** wrappers.

Translation quality – spoken vs written Malayalam. Solved with **text normalization**.

No Raspberry Pi – budget constraint. Pivoted to **Docker simulation** (2 CPU, 4GB RAM).

15 Future Work

45s

Sprint 5 remaining: benchmarking, Docker finalization, response caching.

Sprint 6: unit tests (80%+ coverage), integration tests, docs, final demo.

Post-project: Raspberry Pi deployment, wake word detection, smart home via MQTT, mobile app, RAG knowledge base.

16 Expected Outcome

45s

- **<3 seconds** end-to-end latency
- **93%** translation accuracy
- **11 Indian languages**
- **Zero** cloud dependency

Benefits: **digital inclusion, data sovereignty, zero operational cost, reproducible research** for other low-resource languages.

17 Conclusion

45s

Built a **complete voice AI pipeline** – ASR → Translation → LLM → Agents → TTS – under 3s latency, 11 languages.

State-of-the-art: 2.94% WER (ASR), ~93% translation accuracy, natural responses (Qwen3).

Intelligent agents via LangGraph for diverse user intents.

87% complete, 99/114 story points, on track for Sprint 6 delivery.

This proves **local-first, privacy-preserving AI** can match cloud solutions at zero cost.

18 References

15s

Key research: **IndicTrans2, Whisper, Pingala V1, MMS, Qwen3**. References listed on slide for review.

That concludes my interim presentation. Voice is the most natural interface – by bringing AI to regional languages on edge devices, we can bridge the digital divide for hundreds of millions. **Thank you.** I'm happy to take any questions.

Prepared Q&A

Why not just use Google Assistant or Alexa?

They don't support Malayalam, require constant internet, send voice data to cloud servers, and charge API fees. Ours is **fully offline, private, and free**.

Why Pingala V1 over Whisper?

2.94% WER vs ~5.8% for Whisper on Indian languages. Designed specifically for Indic speech, runs efficiently on CPU.

Why Qwen3 and not GPT or Llama?

Best **performance-to-size ratio**. 0.6B–4B fits edge constraints. llama.cpp enables GGUF quantized inference on CPU. Zero API cost.

How do you handle untested languages?

IndicTrans2 supports all **22 scheduled Indian languages**. We validated 11. Architecture is language-agnostic – no code changes needed.

What's the difference between the agents?

Info Agent = Q&A. **Task Agent** = to-dos/reminders. **Chat Agent** = conversation. **Smart Home Agent** = IoT control. LangGraph routes automatically by intent.

Why Docker instead of Raspberry Pi?

Budget. Docker with **2 CPUs, 4GB RAM** limits accurately simulates Pi 5. Code and models are identical – only hardware differs. Deployment is straightforward when a Pi is available.

What about real-time performance?

~1.5s end-to-end: ASR 480ms, translation 250ms, LLM 340ms, TTS 380ms. Caching reduces repeats to **<200ms**.

How is this different from Bhashini?

Bhashini is a **cloud API gateway** – requires internet, no LLM, no agents. We run **completely offline** with intelligent task routing.